

NỘI DUNG

LỜI NÓI ĐẦU	2
I. GIỚI THIỆU CHUNG	3
1.1. Lý do chọn đề tài.....	3
1.2. Mục tiêu thực tập	4
1.3. Phạm vi và nội dung thực tập.....	5
1.4. Phương pháp thực hiện	5
II. TỔNG QUAN VỀ ĐƠN VỊ THỰC TẬP	6
2.1. Thông tin chung về công ty.....	6
2.2. Cơ cấu tổ chức.....	6
2.3. Lĩnh vực CNTT mà công ty đang triển khai.....	7
2.4. Môi trường làm việc thực tế.....	9
III. NỘI DUNG THỰC TẬP	10
3.1. Tìm hiểu về Python và Pygame	10
3.2. Lập trình game 2D bằng Pygame.....	16
3.3. Tìm hiểu ngôn ngữ C# và Unity Editor	25
3.4. Lập trình game bằng Unity và C#.....	31
IV. NHẬN XÉT – ĐÁNH GIÁ – KIẾN NGHỊ.....	58
4.1. Những điều học được sau thực tập.....	58
4.2. Điểm mạnh và điểm yếu bản thân.....	58
4.3. Góp ý cải tiến cho công ty	59
4.4. Hướng phát triển đề tài/sản phẩm thực tập sau này	59
V. TÀI LIỆU THAM KHẢO.....	59

LỜI NÓI ĐẦU

Trong quá trình thực tập tại Công ty IEC Games, em đã có cơ hội được tiếp cận và làm việc với hai công cụ phát triển game đầy mạnh mẽ là Unity và Pygame. Mỗi công cụ đều mang đến những trải nghiệm và bài học quý giá, giúp em mở rộng hiểu biết về lĩnh vực phát triển game.

Unity, một công cụ phát triển game đa nền tảng, đã cho em thấy được sức mạnh của một engine hiện đại với đầy đủ các tính năng từ đồ họa 2D, 3D, hệ thống vật lý, âm thanh cho đến các công cụ hỗ trợ phát triển VR và AR. Việc sử dụng C# làm ngôn ngữ lập trình chính giúp em rèn luyện kỹ năng lập trình hướng đối tượng và hiểu sâu hơn về quy trình xây dựng một game hoàn chỉnh. Unity không chỉ cung cấp một môi trường phát triển trực quan mà còn mang đến khả năng tối ưu hóa hiệu suất và triển khai sản phẩm lên nhiều nền tảng khác nhau.

Bên cạnh đó, Pygame - một thư viện lập trình game bằng Python - đã giúp em hiểu rõ hơn về các nguyên tắc cốt lõi trong phát triển game. Mặc dù không có đầy đủ tính năng như một game engine hoàn chỉnh, Pygame dạy em cách xây dựng các thành phần game từ những yếu tố cơ bản nhất, từ xử lý sự kiện, vòng lặp game đến quản lý đối tượng và tài nguyên. Việc làm việc với Pygame giúp em nâng cao kỹ năng lập trình Python và hiểu sâu hơn về cách vận hành của một game ở mức độ nguyên lý.

Qua việc sử dụng cả hai công cụ, em không chỉ học được cách lựa chọn công nghệ phù hợp cho từng dự án mà còn phát triển được tư duy linh hoạt trong việc giải quyết vấn đề. Những trải nghiệm này đã trang bị cho em nền tảng vững chắc để tiếp tục học hỏi và phát triển trong lĩnh vực thiết kế và phát triển game.

Em xin chân thành cảm ơn Công ty IEC Games đã tạo điều kiện cho em thực tập và trải nghiệm môi trường làm việc chuyên nghiệp. Đặc biệt, em gửi lời cảm ơn sâu sắc đến thầy Cảnh - người đã trực tiếp hướng dẫn, chỉ bảo tận tình và hỗ trợ em trong suốt quá trình học hỏi và làm việc tại công ty.

I. GIỚI THIỆU CHUNG

1.1. Lý do chọn đề tài

Ngành công nghiệp game hiện nay đang phát triển với tốc độ mạnh mẽ, không chỉ dừng lại ở lĩnh vực giải trí mà còn trở thành một ngành công nghiệp mang lại giá trị kinh tế to lớn. Game có sức hút đặc biệt đối với mọi lứa tuổi và đang dần trở thành một trong những lĩnh vực công nghệ thu hút sự quan tâm của cả nhà phát triển phần mềm, nhà nghiên cứu cũng như cộng đồng người chơi.

Đối với sinh viên ngành công nghệ thông tin, việc phát triển game không chỉ là một thử thách về tư duy lập trình mà còn là cơ hội để rèn luyện kỹ năng giải quyết vấn đề, tư duy logic và khả năng sáng tạo. Game là sự kết hợp của nhiều yếu tố như: lập trình, xử lý đồ họa, âm thanh, thiết kế tương tác và quản lý dữ liệu. Chính sự đa dạng này giúp cho việc làm game trở thành một môi trường học tập toàn diện, gắn lý thuyết với thực tiễn.

Trong quá trình học tập, em nhận thấy rằng để xây dựng một trò chơi hoàn chỉnh, cần phải hiểu rõ từ những khái niệm cơ bản nhất đến những công cụ chuyên nghiệp hơn. Do đó, em lựa chọn đề tài **“Nghiên cứu và xây dựng game Platform 2D bằng Pygame và Unity”**. Lý do là:

- **Pygame** là một thư viện Python đơn giản, thích hợp cho việc học và nắm vững những nguyên lý cơ bản của phát triển game 2D như vòng lặp game, xử lý sự kiện, va chạm, đồ họa và âm thanh.
- **Unity** là một engine chuyên nghiệp, cung cấp nhiều công cụ mạnh mẽ cho việc phát triển game 2D và 3D, phù hợp để xây dựng một sản phẩm game có tính hoàn chỉnh và chuyên nghiệp hơn.

Việc kết hợp cả hai công cụ này giúp em vừa nắm chắc nền tảng cơ bản thông qua Pygame, vừa áp dụng kiến thức vào thực tế với Unity để tạo ra một sản phẩm game Platform 2D hoàn chỉnh. Đây là một quá trình học tập mang tính hệ thống, từ lý thuyết cơ bản đến thực hành ứng dụng, giúp em củng cố kiến thức và phát triển kỹ năng chuyên môn.

1.2. Mục tiêu thực tập

Mục tiêu chính của đợt thực tập là rèn luyện kỹ năng lập trình, áp dụng kiến thức đã học vào thực tế và tìm hiểu sâu hơn về lĩnh vực phát triển game. Cụ thể:

- **Về kiến thức:**

- Hiểu rõ các nguyên lý cơ bản của phát triển game 2D thông qua Pygame: vòng lặp game, xử lý sự kiện, hoạt ảnh, va chạm.
- Làm quen và nắm được các tính năng quan trọng của Unity trong phát triển game Platform 2D như hệ thống tilemap, animation, xử lý va chạm, camera, UI và âm thanh.
- Hiểu quy trình phát triển một sản phẩm phần mềm/game: từ khâu xây dựng ý tưởng, phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến hoàn thiện sản phẩm.

- **Về kỹ năng:**

- Vận dụng ngôn ngữ lập trình (Python, C#) và công cụ hỗ trợ (Unity, Pygame, GitHub, Visual Studio/VS Code) để phát triển sản phẩm.
- Rèn luyện kỹ năng phân tích vấn đề, xử lý lỗi và tối ưu chương trình.
- Nâng cao khả năng tự học, nghiên cứu tài liệu và áp dụng vào thực tế.
- Rèn luyện kỹ năng làm việc nhóm, trao đổi và tiếp thu ý kiến từ người hướng dẫn.

- **Về thái độ:**

- Hình thành tác phong làm việc nghiêm túc, chủ động, có trách nhiệm với công việc được giao.
- Xây dựng thói quen kiên nhẫn, cẩn thận trong quá trình lập trình và thử nghiệm sản phẩm.
- Có tinh thần cầu tiến, sẵn sàng học hỏi và cải thiện bản thân.

1.3. Phạm vi và nội dung thực tập

- **Phạm vi:** Đề tài tập trung nghiên cứu và thực hành phát triển game 2D theo thể loại Platform. Trong đó:
 - Với **Pygame**, phạm vi chủ yếu là học tập, thử nghiệm các cơ chế cơ bản của một game 2D.
 - Với **Unity**, phạm vi mở rộng sang việc xây dựng một trò chơi Platform 2D hoàn chỉnh với các chức năng như: điều khiển nhân vật, hệ thống chương ngại vật, tính toán điểm số, giao diện người dùng, âm thanh và hiệu ứng.
- **Nội dung chính:**
 - Tìm hiểu quy trình phát triển phần mềm/game tại công ty thực tập.
 - Nghiên cứu Pygame để nắm vững các nguyên lý cơ bản trong phát triển game.
 - Tìm hiểu Unity và các tính năng hỗ trợ xây dựng game Platform 2D.
 - Thiết kế, lập trình và kiểm thử game Platform 2D.
 - So sánh sự khác biệt giữa Pygame và Unity trong quá trình phát triển game.
 - Hoàn thiện sản phẩm demo và viết báo cáo tổng kết.

1.4. Phương pháp thực hiện

- **Nghiên cứu tài liệu:** Tham khảo tài liệu chính thức của Unity và Pygame, cùng các giáo trình, khóa học trực tuyến, và tài liệu do công ty cung cấp.
- **Quan sát – học hỏi:** Quan sát quy trình làm việc thực tế tại công ty, cách các lập trình viên triển khai ý tưởng và xử lý vấn đề trong phát triển game.
- **Thực hành:**
 - Viết code thử nghiệm trên Pygame để nắm các nguyên tắc cơ bản.
 - Xây dựng dự án chính trên Unity để phát triển game Platform 2D hoàn chỉnh.
 - Kiểm thử, tìm lỗi và tối ưu sản phẩm.

- **Trao đổi – thảo luận:** Chủ động hỏi ý kiến người hướng dẫn và trao đổi với đồng nghiệp để tìm ra hướng giải quyết cho các vấn đề kỹ thuật gặp phải.
- **Tổng hợp – báo cáo:** Ghi chép quá trình học tập, thực hiện và hoàn thiện sản phẩm. Sau đó tổng hợp thành báo cáo thực tập để đánh giá kết quả.

II. TỔNG QUAN VỀ ĐƠN VỊ THỰC TẬP

2.1. Thông tin chung về công ty

- Tên đầy đủ: **Công ty Cổ phần IEC Games**
- Tên quốc tế / viết tắt: IEC Games Joint Stock Company
- Mã số thuế: 0109964115
- Địa chỉ trụ sở: Tầng 3, nhà 25T1 – N05, Hoàng Đạo Thúy, Trung Hòa, Cầu Giấy, Hà Nội.
- Quy mô: khoảng **100-200 nhân viên**
- Loại hình: Công ty cổ phần, hoạt động trong lĩnh vực phát triển game di động, kinh doanh phần mềm & giải trí số. IEC Games là một bộ phận/nhánh thuộc Công ty Cổ phần Đầu tư Giáo dục và Truyền Thông Quốc tế (IEC Corporation)



2.2. Cơ cấu tổ chức

- Công ty chia thành nhiều **studio/team sản xuất game**. Ví dụ có team **Winter Wolf** là một trong các studio con của IEC Games.
- Các phòng ban chức năng:

1. **Phòng phát triển sản phẩm/game (Development):** dev, lập trình viên (Unity, Pygame hoặc các thư viện khác), tester.
2. **Phòng thiết kế:** Game Designer, 2D Artist, UI/UX Designer.
3. **Phòng QA / kiểm thử:** kiểm thử game, sửa lỗi, đảm bảo chất lượng sản phẩm.
4. **Phòng sản xuất nội dung / sáng tạo:** âm thanh, hình ảnh – nếu có.
5. **Phòng xuất bản / marketing / phân phối:** chịu trách nhiệm phát hành game lên các platform (App Store, Google Play...), marketing app, theo dõi chỉ số người dùng.
6. **Phòng nhân sự / hỗ trợ / quản lý chung:** nhân sự, hành chính, tài chính, hỗ trợ học tập & đào tạo nhân viên.

2.3. Lĩnh vực CNTT mà công ty đang triển khai

- Phát triển **game di động (mobile games)**, đặc biệt dòng **Hyper Casual** và **Puzzle**.
- Xuất bản game và phát hành game trên các nền tảng mobile (Google Play, App Store) cho người dùng trong nước và quốc tế.
- Đổi mới liên tục trong thiết kế sản phẩm để thu hút người dùng, cải tiến trải nghiệm người dùng.
- Có các team nội bộ chuyên trách để đảm bảo phát triển game từ khâu thiết kế, lập trình đến thử nghiệm và release.



2.4. Môi trường làm việc thực tế

- Làm việc trong môi trường trẻ trung, năng động, có nhiều cơ hội học hỏi: do công ty tuyển nhiều vị trí fresher, thực tập sinh.
- Được hỗ trợ học tập, đào tạo: có gần như hỗ trợ nhân viên mới, mentors (người hướng dẫn) để phát triển kỹ năng chuyên môn.
- Tính linh hoạt cao: môi trường có thể hỗ trợ cho thực tập sinh làm việc dưới sự hướng dẫn, được tham gia trực tiếp vào các dự án nhỏ hoặc phần việc thực tế.
- Văn hóa công ty có nhiều ưu đãi:
 - Không máy chấm công



III. NỘI DUNG THỰC TẬP

3.1. Tìm hiểu về Python và Pygame

3.1.1. Ngôn ngữ lập trình Python

Python là ngôn ngữ lập trình bậc cao, thông dịch, được Guido van Rossum giới thiệu lần đầu vào năm 1991. Cái tên "Python" không liên quan đến loài rắn mà được lấy cảm hứng từ chương trình hài kịch nổi tiếng "Monty Python's Flying Circus", thể hiện triết lý thiết kế đề cao sự đơn giản, rõ ràng và vui vẻ trong lập trình.

Đặc điểm nổi bật của Python:

- **Cú pháp đơn giản, dễ đọc:** Python sử dụng thụt lề để phân tách khối lệnh thay vì các dấu ngoặc như nhiều ngôn ngữ khác, giúp code trở nên sáng sủa và dễ bảo trì.
- **Đa nền tảng:** Python chạy được trên Windows, macOS, Linux mà không cần thay đổi code.
- **Hỗ trợ đa mô hình lập trình:** Python cho phép lập trình hướng đối tượng (OOP), lập trình hàm (functional programming) và lập trình thủ tục (procedural programming).
- **Hệ thống thư viện phong phú:** Thông qua PyPI (Python Package Index), Python cung cấp hàng trăm ngàn thư viện cho nhiều lĩnh vực như phân tích dữ liệu (Pandas, NumPy), machine learning (Scikit-learn, TensorFlow), phát triển web (Django, Flask) và game (Pygame).
- **Cộng đồng hỗ trợ mạnh mẽ:** Python có tài liệu phong phú, diễn đàn sôi động và số lượng người dùng đông đảo trên toàn thế giới.

Ví dụ minh họa:

```
# In ra các số chẵn từ 0 đến 10  
  
for i in range(0, 11, 2):  
  
    print(i)
```

Kết quả:

0

2

4

6

8

10

Code Python rất dễ hiểu ngay cả với người mới bắt đầu, nhờ cú pháp gần gũi với ngôn ngữ tự nhiên.

3.1.2. Thư viện Pygame

Pygame là một thư viện mã nguồn mở được xây dựng dựa trên thư viện SDL (Simple DirectMedia Layer), cung cấp các công cụ đơn giản và hiệu quả để phát triển game 2D. Ra đời năm 2000, Pygame nhanh chóng trở thành lựa chọn phổ biến cho giáo dục và phát triển game nghiệp dư nhờ tính dễ sử dụng và miễn phí.

Các module chính của Pygame:

- `pygame.display`: Quản lý cửa sổ game và hiển thị đồ họa.
- `pygame.event`: Xử lý các sự kiện từ bàn phím, chuột, joystick.
- `pygame.image`: Tải và hiển thị ảnh.
- `pygame.mixer`: Phát âm thanh và nhạc nền.
- `pygame.draw`: Vẽ các hình học cơ bản.
- `pygame.sprite`: Hỗ trợ tạo và quản lý các đối tượng game.

Ưu điểm của Pygame:

- Dễ học, dễ sử dụng, phù hợp cho người mới.
- Nhẹ, chạy tốt trên nhiều hệ điều hành.
- Cung cấp đầy đủ công cụ cần thiết để xây dựng game 2D cơ bản.
- Tài liệu phong phú, cộng đồng hỗ trợ tích cực.

Hạn chế:

- Hiệu suất không cao bằng các game engine như Unity hoặc Unreal.
- Không hỗ trợ đồ họa 3D chuyên nghiệp.
- Thiếu các công cụ biên tập trực quan (phải code hoàn toàn bằng tay).

3.1.3. Ví dụ minh họa chi tiết với Pygame

Dưới đây là một ví dụ chi tiết hơn về việc sử dụng Pygame để tạo một game đơn giản, trong đó người chơi điều khiển một nhân vật di chuyển và thu thập vật phẩm.

```
import pygame
```

```
import random
```

```
# Khởi tạo Pygame
```

```
pygame.init()
```

```
# Cài đặt cửa sổ game
```

```
WIDTH, HEIGHT = 800, 600
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("Thu thập vật phẩm")
```

```
# Màu sắc
```

```
WHITE = (255, 255, 255)
```

```
RED = (255, 0, 0)
```

```
GREEN = (0, 255, 0)
```

```
BLUE = (0, 0, 255)
```

```
# Người chơi
```

```

player_size = 50

player_x = WIDTH // 2 - player_size // 2

player_y = HEIGHT - player_size - 10

player_speed = 5


# Vật phẩm

item_size = 30

items = []

for _ in range(5):

    item_x = random.randint(0, WIDTH - item_size)

    item_y = random.randint(-100, -40)

    items.append([item_x, item_y, BLUE])


# Score

score = 0

font = pygame.font.SysFont(None, 36)


# Vòng lặp game

clock = pygame.time.Clock()

running = True

while running:

    # Xử lý sự kiện

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

```

```

    running = False

    # Di chuyển người chơi
    keys = pygame.key.get_pressed()

    if keys[pygame.K_LEFT] and player_x > 0:
        player_x -= player_speed

    if keys[pygame.K_RIGHT] and player_x < WIDTH - player_size:
        player_x += player_speed

    # Di chuyển vật phẩm
    for item in items:
        item[1] += 3 # Rơi xuống

        if item[1] > HEIGHT: # Rơi ra ngoài màn hình
            item[0] = random.randint(0, WIDTH - item_size)
            item[1] = random.randint(-100, -40)

    # Kiểm tra va chạm
    if (player_x < item[0] + item_size and
        player_x + player_size > item[0] and
        player_y < item[1] + item_size and
        player_y + player_size > item[1]):
        score += 1

        item[0] = random.randint(0, WIDTH - item_size)
        item[1] = random.randint(-100, -40)

```

```

# Vẽ

screen.fill(WHITE)

pygame.draw.rect(screen, RED, (player_x, player_y, player_size, player_size))

for item in items:

    pygame.draw.rect(screen, item[2], (item[0], item[1], item_size, item_size))


# Hiển thị điểm

score_text = font.render(f"Điểm: {score}", True, (0, 0, 0))

screen.blit(score_text, (10, 10))


pygame.display.flip()

clock.tick(60)


pygame.quit()

```

Trong ví dụ này, chúng ta đã tạo một game đơn giản với các tính năng:

- Điều khiển nhân vật bằng phím mũi tên
- Vật phẩm rơi ngẫu nhiên từ trên xuống
- Tính điểm khi người chơi chạm vào vật phẩm
- Hiển thị điểm trên màn hình

3.1.4. Kết luận

Python và Pygame tạo thành một bộ công cụ mạnh mẽ cho việc học tập và phát triển game 2D. Mặc dù không thể so sánh với các game engine chuyên nghiệp như Unity hay

Unreal về mặt hiệu năng và tính năng, sự kết hợp này cung cấp một môi trường lý tưởng để người mới làm quen với các khái niệm cơ bản trong phát triển game.

Kiến thức thu được từ việc học Python và Pygame đã tạo nền tảng vững chắc để em tiếp tục học hỏi và làm việc với các công nghệ phức tạp hơn trong tương lai, đồng thời giúp em phát triển tư duy logic và kỹ năng giải quyết vấn đề - những yếu tố quan trọng trong sự nghiệp phát triển game.

3.2. Lập trình game 2D bằng Pygame

3.2.1. Mô tả nhiệm vụ

Trong quá trình thực tập tại Công ty IEC Games, em được giao nhiệm vụ phát triển một game platform 2D hoàn chỉnh sử dụng thư viện Pygame. Nhiệm vụ bao gồm xây dựng các chức năng cơ bản của game như di chuyển nhân vật, hệ thống va chạm, enemy AI, thu thập item, quản lý level và thiết kế overworld.

3.2.2. Cách thực hiện

Dự án được phát triển với kiến trúc module hóa, chia thành các thành phần độc lập:

a. Cấu trúc project:

- audio: Chứa các file âm thanh của game
- graphics: Chứa các hình ảnh đồ họa của game
- data: Chứa các file liên quan đến map của game
- code: Chứa code của game
 - main.py: Khởi tạo game, quản lý trạng thái, chuyển đổi giữa các level
 - settings.py: Chứa các hằng số, cấu hình game
 - data.py: Quản lý dữ liệu người chơi (health, coins)
 - level.py: Xử lý logic của từng level
 - overworld.py: Quản lý bản đồ overworld
 - player.py: Điều khiển nhân vật chính
 - enemies.py: AI cho các enemy (Tooth, Shell, Pearl)
 - sprites.py: Định nghĩa các loại sprite và animation

- groups.py: Quản lý nhóm sprite và camera
- ui.py: Giao diện người dùng (health, coins)
- timer.py: Hệ thống timer cho game
- support.py: Hỗ trợ import assets

b. Di chuyển nhân vật:

Trong player.py

def input(self):

keys = pygame.key.get_pressed()

if not self.timers['wall jump'].active:

if keys[pygame.K_RIGHT]:

self.direction.x = 1

self.facing_right = True

if keys[pygame.K_LEFT]:

self.direction.x = -1

self.facing_right = False

if keys[pygame.K_SPACE]:

self.jump = True

def move(self, dt):

Horizontal movement

*self.hitbox_rect.x += self.direction.x * self.speed * dt*

self.collision('horizontal')

Vertical movement + gravity

```
self.direction.y += self.gravity / 2 * dt
```

```
self.hitbox_rect.y += self.direction.y * dt
```

```
self.direction.y += self.gravity / 2 * dt
```

c. Hệ thống va chạm:

```
def collision(self, axis):
```

```
    for sprite in self.collision_sprites:
```

```
        if sprite.rect.collidect(self.hitbox_rect):
```

```
            if axis == 'horizontal':
```

```
                if self.hitbox_rect.left <= sprite.rect.right and int(self.old_rect.left) >=
int(sprite.old_rect.right):
```

```
                    self.hitbox_rect.left = sprite.rect.right
```

```
                if self.hitbox_rect.right >= sprite.rect.left and int(self.old_rect.right) <=
int(sprite.old_rect.left):
```

```
                    self.hitbox_rect.right = sprite.rect.left
```

d. Animation system:

```
def animate(self, dt):
```

```
    self.frame_index += ANIMATION_SPEED * dt
```

```
    self.image=self.frames[self.state][int(self.frame_index%len(self.frames[self.state]))]
```

```
    self.image = self.image if self.facing_right else pygame.transform.flip(self.image, True,
False)
```

e. Enemy AI:

```
# Trong enemies.py
```

```
class Tooth(AnimatedSprite):
```

```
    def __init__(self, pos, frames, groups, collision_sprites):
```

```
        super().__init__(pos, frames, groups)
```

```
self.collision_sprites = collision_sprites
```

```
self.direction = vector(-1, 0)
```

```
self.speed = 120
```

```
def move(self, dt):
```

```
    self.rect.x += self.direction.x * self.speed * dt
```

```
    self.check_collision()
```

```
def check_collision(self):
```

```
    for sprite in self.collision_sprites:
```

```
        if sprite.rect.colliderect(self.rect):
```

```
            self.direction.x *= -1
```

```
            break
```

f. Item system:

```
class Item(AnimatedSprite):
```

```
    def __init__(self, item_type, pos, frames, groups, data):
```

```
        super().__init__(pos, frames, groups)
```

```
        self.item_type = item_type
```

```
        self.data = data
```

```
    def activate(self):
```

```
        if self.item_type == 'gold':
```

```
            self.data.coins += 5
```

```
        elif self.item_type == 'silver':
```

```

        self.data.coins += 1

    elif self.item_type == 'potion':

        self.data.health += 1

```

g. Camera system:

```

# Trong groups.py

def draw(self, target_pos, dt):

    self.offset.x = -(target_pos[0] - WINDOW_WIDTH / 2)

    self.offset.y = -(target_pos[1] - WINDOW_HEIGHT / 2)

    for sprite in sorted(self, key=lambda sprite: sprite.z):

        offset_pos = sprite.rect.topleft + self.offset

        self.display_surface.blit(sprite.image, offset_pos)

```

h. Overworld system:

```

# Trong overworld.py

def setup(self, tmx_map, overworld_frames):

    # Tạo nodes và paths

    for obj in tmx_map.get_layer_by_name('Nodes'):

        if obj.name == 'Node':

            available_paths = {k: v for k, v in obj.properties.items()

                               if k in ('left', 'right', 'up', 'down')}

            Node(pos=(obj.x, obj.y), surf=overworld_frames['path']['node'],

                 groups=(self.all_sprites, self.node_sprites),

                 level=obj.properties['stage'], data=self.data,

                 paths=available_paths)

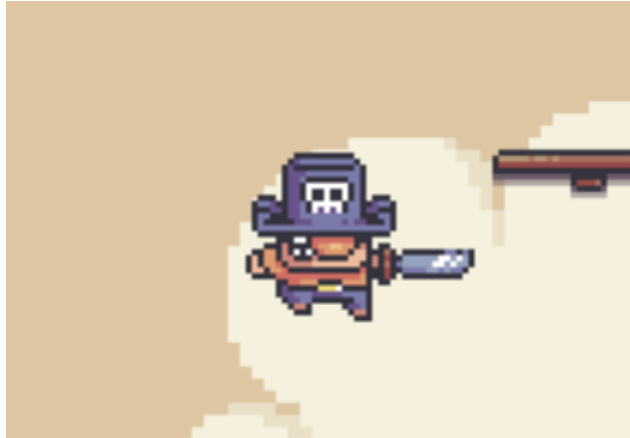
```

3.2.3. Kết quả đạt được

Sau thời gian phát triển, em đã hoàn thành một game platform 2D hoàn chỉnh với các tính năng:

a. Player: với khả năng di chuyển linh hoạt:

- Di chuyển trái/phải với animation mượt mà
- Nhảy và wall jump



- Tấn công với weapon



- Nhận damage và hiệu ứng flicker



b. Hệ thống enemy đa dạng:

- Tooth: Di chuyển tới lui và gây damage



- Shell: Bắn pearl về phía player
- Pearl: Đạn được bắn từ Shell



c. Hệ thống item và collectibles:

- Coin (vàng, bạc, kim cương) để tăng điểm



- Potion để hồi health

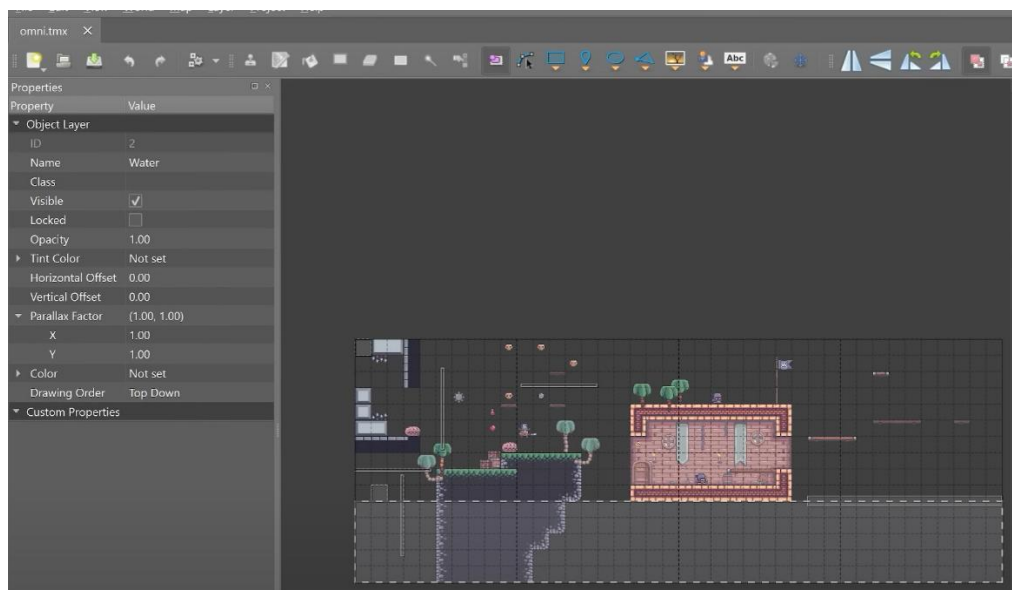


- Hiệu ứng particle khi thu thập item



b. Thiết kế level linh hoạt:

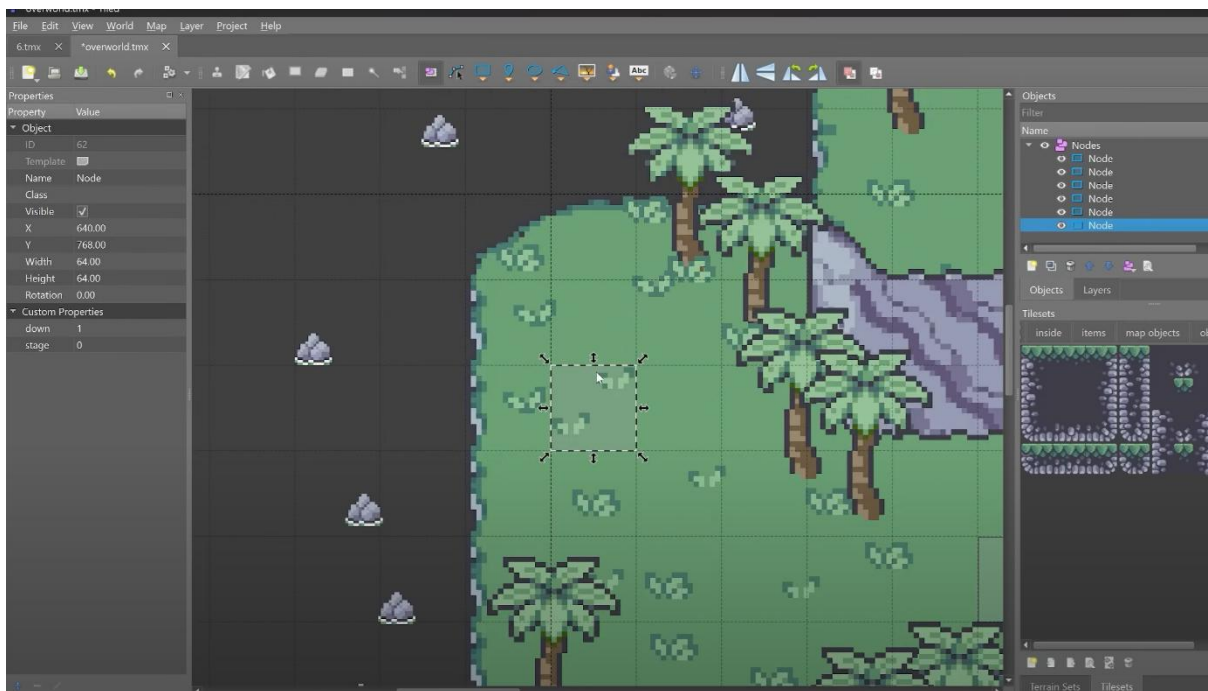
- Hỗ trợ Tiled Map Editor (.tmx)
- Nhiều layer (BG, Terrain, Platforms, Objects)
- Moving platforms và traps



c. Overworld system:

- Map tổng quan với các node level

- Path system để di chuyển giữa các level
- Level locking/unlocking



d. Giao diện người dùng:

- Hiển thị health dưới dạng hearts
- Hiển thị số coins thu thập được
- Animation cho UI elements



e. Âm thanh và hiệu ứng:

- Background music

- Sound effects cho jump, attack, collect items
- Particle effects

3.2.4. Kết luận

Qua dự án này, em đã áp dụng thành công các kiến thức về Pygame để xây dựng một game platform 2D hoàn chỉnh. Dự án không chỉ rèn luyện kỹ năng lập trình Python mà còn phát triển tư duy giải quyết vấn đề trong game development, bao gồm xử lý va chạm, animation, AI cho enemy, và quản lý trạng thái game.

Kiến thức thu được từ dự án này là nền tảng vững chắc để em tiếp tục phát triển các game phức tạp hơn trong tương lai, đồng thời hiểu sâu hơn về quy trình phát triển game chuyên nghiệp.

3.3. Tìm hiểu ngôn ngữ C# và Unity Editor

Mô tả nhiệm vụ:

Nhiệm vụ này nhằm mục đích làm quen với Unity – một công cụ phát triển game chuyên nghiệp hàng đầu thế giới. Trọng tâm là tìm hiểu về giao diện Unity Editor (Scene, GameObject, Prefab) và học cú pháp cơ bản của ngôn ngữ lập trình C#. Cuối cùng, áp dụng kiến thức đã học để viết script điều khiển nhân vật trong môi trường game 2D.

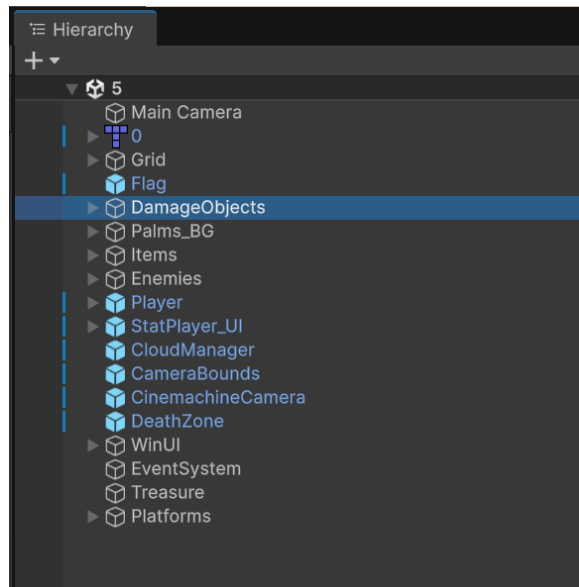
Cách thực hiện:

3.3.1. Cài đặt Unity và tìm hiểu về Unity Editor:

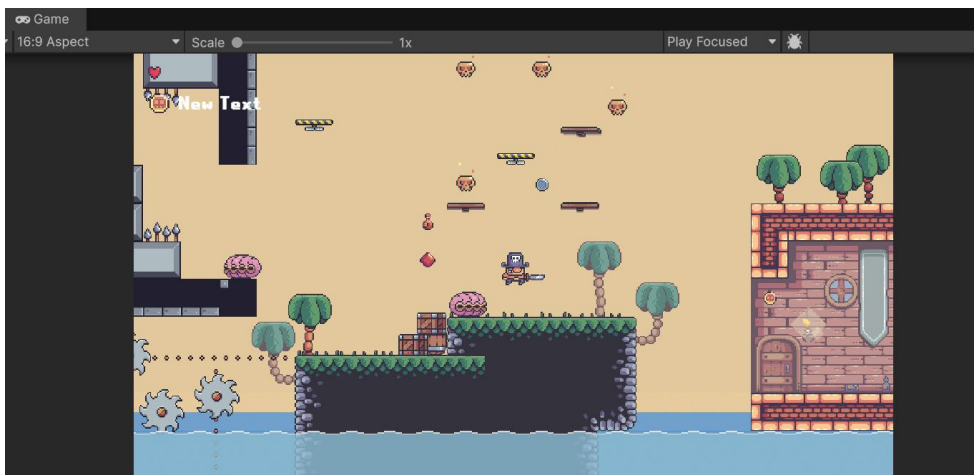
- **Cài đặt:** Tiến hành tải và cài đặt Unity Hub, sau đó cài đặt phiên bản Unity phù hợp cùng với các module cần thiết cho phát triển game 2D.
- **Tìm hiểu giao diện Unity Editor:** Làm quen với bố cục và chức năng của các cửa sổ chính trong Unity, dựa trên tài liệu đã được cung cấp:
 - **A. Thanh công cụ (Toolbar):** Chứa các nút điều khiển chính cho việc thiết kế và chạy mô phỏng.



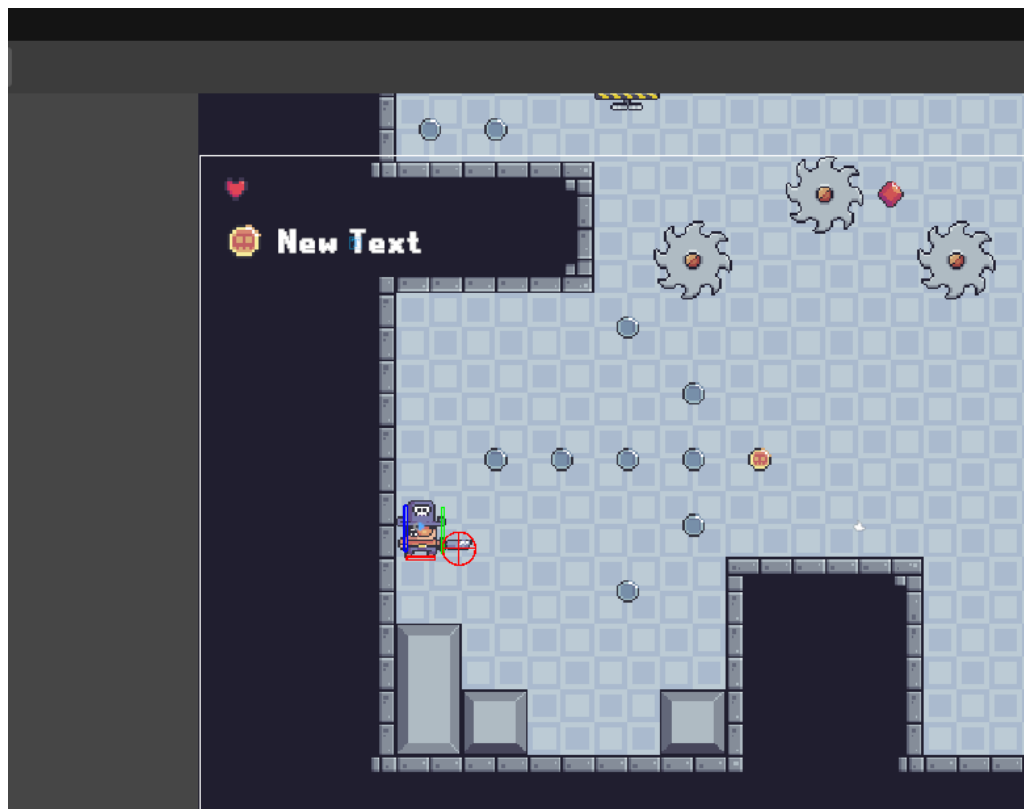
- **B. Cửa sổ phân cấp (Hierarchy):** Hiển thị danh sách tất cả các GameObject đang được sử dụng trong Scene dưới dạng phân cấp cha-con.



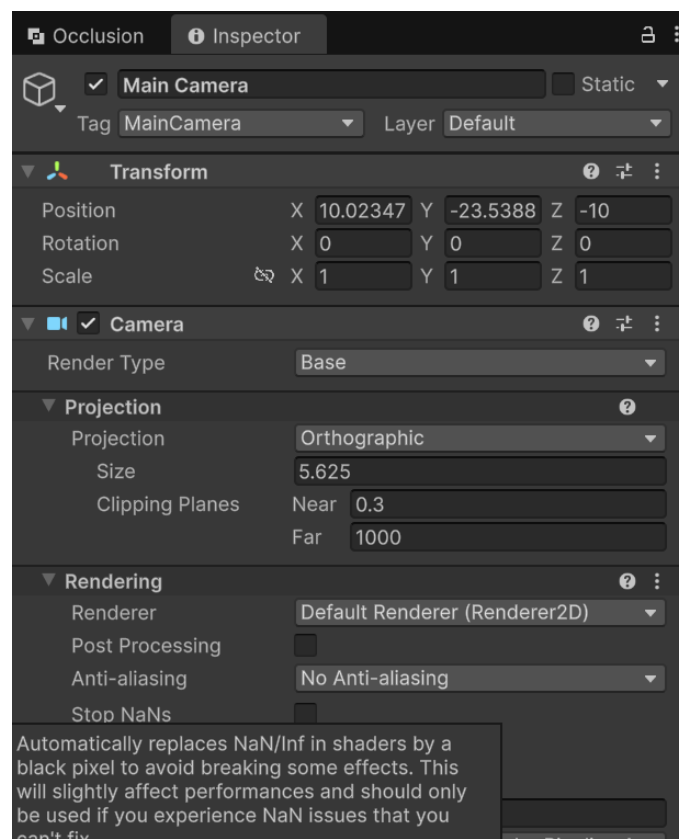
- **C. Cửa sổ mô phỏng (Game View):** Hiển thị kết quả cuối cùng của trò chơi thông qua góc nhìn của các Camera.



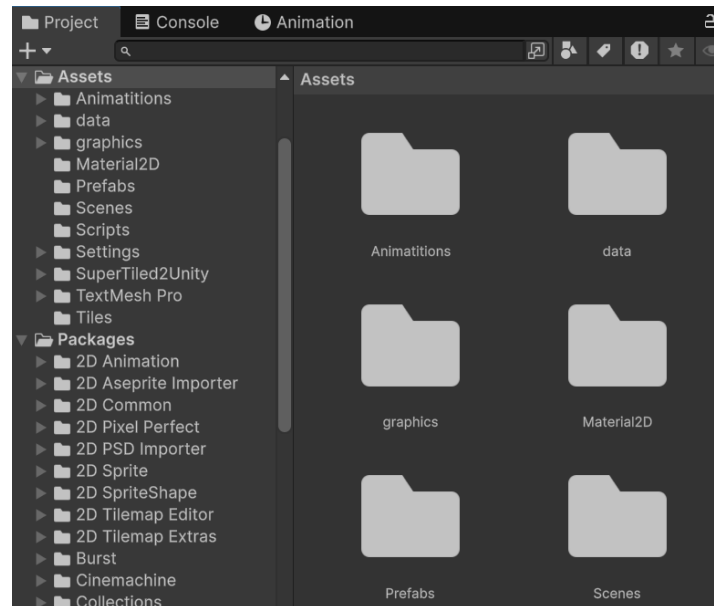
- **D. Cửa sổ thiết kế (Scene View):** Là không gian để trực tiếp tạo, sắp xếp và chỉnh sửa các GameObject và khung cảnh.



- **E. Cửa sổ thuộc tính (Inspector):** Hiển thị và cho phép chỉnh sửa tất cả các thuộc tính, thành phần (Components) của GameObject được chọn.



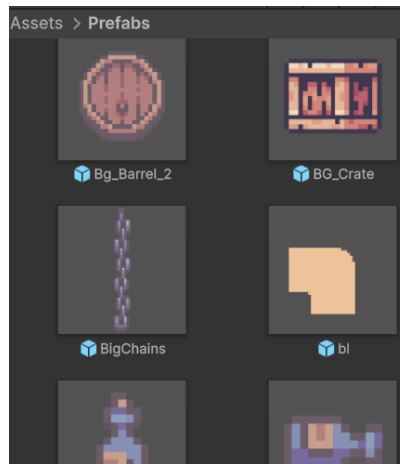
- **F. Cửa sổ dự án (Project):** Là kho lưu trữ tất cả các tài nguyên (Assets) của dự án.



- **G. Thanh trạng thái (Status Bar):** Cung cấp thông tin về các hoạt động đang diễn ra và cảnh báo lỗi.

- **Khái niệm cốt lõi:**

- **Scene:** Là một không gian làm việc, nơi thiết kế các level hoặc màn chơi khác nhau.
- **GameObject:** Là các đối tượng cơ bản tồn tại trong một Scene, có thể là nhân vật, vật cản, camera, nguồn sáng, v.v. Mỗi GameObject có thể chứa nhiều Components (như Transform, Rigidbody2D, Script, v.v.) để định nghĩa thuộc tính và hành vi của nó.
- **Prefab:** Là một dạng Asset đặc biệt, đóng vai trò như một bản mẫu (template) cho một GameObject hoặc một nhóm GameObject. Prefab cho phép tái sử dụng, chỉnh sửa tập trung và khởi tạo các đối tượng trong lúc runtime.



3.3.2. Học cú pháp cơ bản của ngôn ngữ C#:

- Nghiên cứu các đặc trưng cơ bản của C# – ngôn ngữ lập trình chủ đạo trong Unity:
 - **Đơn giản:** C# loại bỏ các tính năng phức tạp có trong C++ và Java (như macro, template, đa kế thừa) nhưng vẫn đảm bảo sức mạnh và linh hoạt.
 - **Hiện đại:** Hỗ trợ các tính năng hiện đại như xử lý ngoại lệ, thu gom rác tự động, và bảo mật mã nguồn.
 - **Hướng đối tượng:** Là ngôn ngữ thuần hướng đối tượng, hỗ trợ đầy đủ bốn tính chất: Tính đóng gói, Tính kế thừa, Tính đa hình và Tính trừu tượng.
 - **Mã nguồn mở & Đa nền tảng:** C# là ngôn ngữ mã nguồn mở và có thể chạy trên nhiều nền tảng như Windows, Linux, macOS.
- Tập trung vào các cú pháp cơ bản như: khai báo biến, kiểu dữ liệu, các cấu trúc điều khiển (if, for, while), cách định nghĩa và sử dụng lớp (class), phương thức (method), và cách kế thừa.

3. Thực hành viết script điều khiển nhân vật bằng C#:

- Tạo một script C# mới trong Unity và gắn vào GameObject của nhân vật.
- Sử dụng các hàm sự kiện có sẵn của Unity như Start() (khởi tạo) và Update() (cập nhật mỗi frame).
- Truy cập và thao tác với các Components (ví dụ: Rigidbody2D, Transform) thông qua lệnh GetComponent<>().

- Viết code để xử lý đầu vào từ người dùng (Input) từ bàn phím hoặc thiết bị khác (ví dụ: các phím mũi tên, phím W, A, S, D).
- Áp dụng các kiến thức vật lý cơ bản trong Unity (như lực, vận tốc) thông qua Rigidbody2D để di chuyển nhân vật một cách mượt mà và xử lý va chạm.

Ví dụ code cơ bản cho di chuyển nhân vật 2D:

```
using UnityEngine;
```

```
public class PlayerMovement : MonoBehaviour
```

```
{
```

```
    public float moveSpeed = 5f; // Tốc độ di chuyển
```

```
    public Rigidbody2D rb; // Tham chiếu đến Rigidbody2D
```

```
    Vector2 movement; // Lưu trữ hướng di chuyển
```

```
    void Update()
```

```
    {
```

```
        // Nhận input từ người chơi trong Update
```

```
        movement.x = Input.GetAxisRaw("Horizontal");
```

```
        movement.y = Input.GetAxisRaw("Vertical");
```

```
    }
```

```
    void FixedUpdate()
```

```
    {
```

```
        // Di chuyển nhân vật trong FixedUpdate để đảm bảo tính đồng bộ với vật lý
```

```

        rb.MovePosition(rb.position + movement * moveSpeed * Time.fixedDeltaTime);
    }
}

```

Kết quả đạt được:

Sau khi hoàn thành nhiệm vụ này, em đã đạt được những kết quả sau:

- **Biết cách tạo và quản lý Project Unity:** Thành thạo việc tạo project mới, import và quản lý các assets.
- **Thao tác thành thạo với giao diện Unity Editor:** Hiểu rõ và sử dụng hiệu quả các cửa sổ làm việc chính như Scene, Hierarchy, Inspector, Project.
- **Nắm vững các khái niệm cơ bản:** Hiểu và vận dụng được các khái niệm then chốt như GameObject, Component, Prefab.
- **Áp dụng kiến thức lập trình C# cơ bản:** Viết được script đơn giản để điều khiển nhân vật, xử lý input và tương tác với physics engine của Unity.
- **Tạo được chuyển động cơ bản cho nhân vật trong game 2D:** Nhân vật có thể di chuyển theo sự điều khiển của người dùng, tạo tiền đề cho việc phát triển các gameplay phức tạp hơn trong tương lai.

Qua nhiệm vụ này, em đã có những bước đầu tiên vững chắc trong việc sử dụng công cụ Unity và ngôn ngữ C# để phát triển game, đồng thời hiểu rõ hơn về quy trình làm việc trong một dự án game thực tế.

3.4. Lập trình game bằng Unity và C#

Mô tả nhiệm vụ:

Tạo ra 1 game platform 2D bằng Unity2D có tên là “Super Pirate World”: nhân vật cướp biển từ vị trí xuất phát ban đầu di chuyển qua các chướng ngại vật, kẻ thù và thu thập vật phẩm để có thể về đích và tìm thấy kho báu:

Cách chơi:

Người chơi sẽ điều khiển nhân vật vượt qua nhiều màn chơi liên tiếp. Để mở khóa màn tiếp theo, người chơi cần hoàn thành màn hiện tại bằng cách di chuyển tới **cột cờ** ở cuối màn.

Trong quá trình chơi, nhân vật sẽ đối mặt với nhiều thử thách và kẻ thù:

- **Tooth:** quái vật có khả năng di chuyển. Khi nhân vật chạm vào sẽ bị trừ máu.
- **Shell:** đứng yên tại chỗ và phóng ra đạn ngọc trai để tấn công người chơi.
- **Chướng ngại vật:** bao gồm những chiếc chông, cưa xoay và những quả chùy gai sắc nhọn treo lơ lửng, cản trở đường đi.

Cơ chế chiến đấu và vật phẩm:

- Người chơi có thể nhấn phím **“J”** để tấn công. Đòn tấn công này có thể **đổi chiều di chuyển của kẻ thù** (ví dụ Tooth) và **đổi hướng bay của đạn**.
- Nếu nhân vật trúng đạn, va chạm với kẻ thù hoặc bẫy, sẽ bị trừ máu.
- Khi nhặt vật phẩm, người chơi có thể nhận được **tiền** hoặc **máu**.
- Khi tích lũy đủ một lượng tiền nhất định, số tiền đó sẽ được quy đổi thành máu.
- Hoàn thành một màn chơi cũng giúp nhân vật hồi thêm 1 máu.
- Nếu rơi xuống đáy biển, nhân vật sẽ bị trừ máu và phải chơi lại màn chơi.

Điều kiện thất bại:

- Nếu nhân vật mất hết máu, sẽ chết và phải chơi lại từ đầu.

Chiến thắng:

- Khi vượt qua **màn chơi cuối cùng**, người chơi sẽ giành chiến thắng chung cuộc và mở khóa **kho báu cướp biển**.

Cách thực hiện:

3.4.1 Thiết kế bản đồ



Tile Palette trong Unity 2D là một công cụ hỗ trợ mạnh mẽ cho việc xây dựng môi trường game. Tile (ô gạch) là những hình ảnh nhỏ được dùng để tạo nên **Tilemap**, tức là bản đồ gồm nhiều ô lưới, giúp thiết kế thế giới 2D một cách nhanh chóng và trực quan.

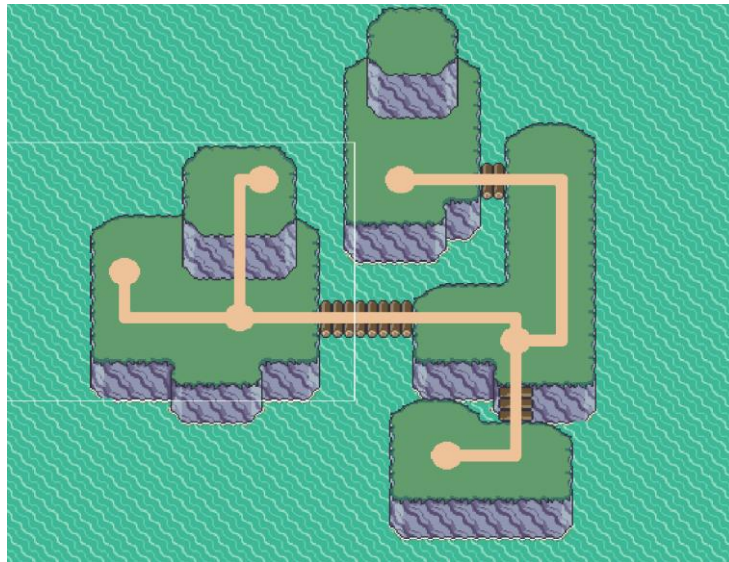
Để sử dụng **Tile Palette**, người phát triển có thể mở bằng đường dẫn **Window > 2D > Tile Palette**. Tại đây, toàn bộ các Tile Palette đã tạo trong dự án sẽ được hiển thị. Người dùng cũng có thể tạo một Tile Palette mới bằng cách nhấn nút **New Palette** và đặt tên theo ý muốn. Ngoài những tile tĩnh thông thường, Unity còn cho phép tạo **Animated Tile** để thiết kế các đối tượng có chuyển động, tăng tính sinh động cho bản đồ.

Việc thêm tile vào Tile Palette khá đơn giản: chỉ cần kéo – thả hình ảnh từ thư mục dự án vào cửa sổ Tile Palette, hoặc sử dụng nút **Add Tile** để chọn trực tiếp tệp từ máy tính. Sau khi được thêm, tile có thể được sắp xếp, chỉnh sửa và dùng để vẽ bản đồ bằng công cụ **Tilemap Brush**.

Khi đã hoàn thiện, các **Tilemap** sẽ trở thành nền tảng của màn chơi, định hình không gian di chuyển của nhân vật, vị trí chương ngại vật, cũng như khung cảnh tổng thể của trò chơi.

Các bản đồ sau khi được thiết kế bằng Tile Palette:

1. Overworld:



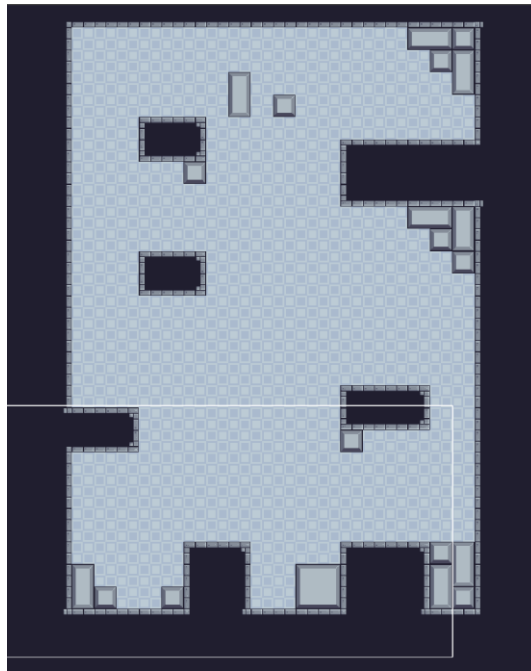
2. *Level 0:*



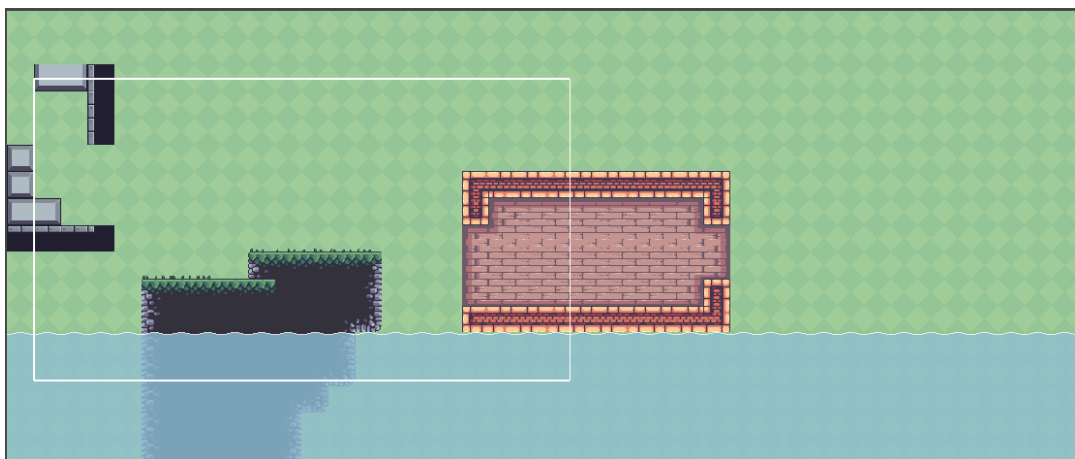
3. *Level 1:*



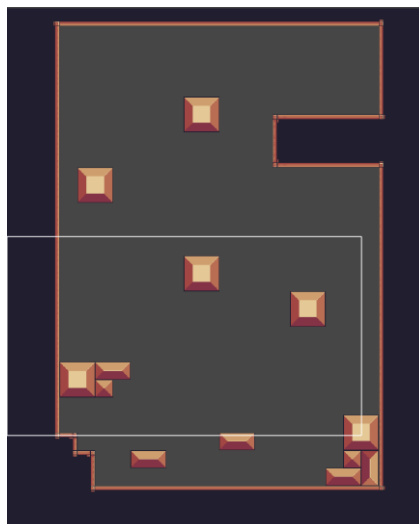
4. *Level 2:*



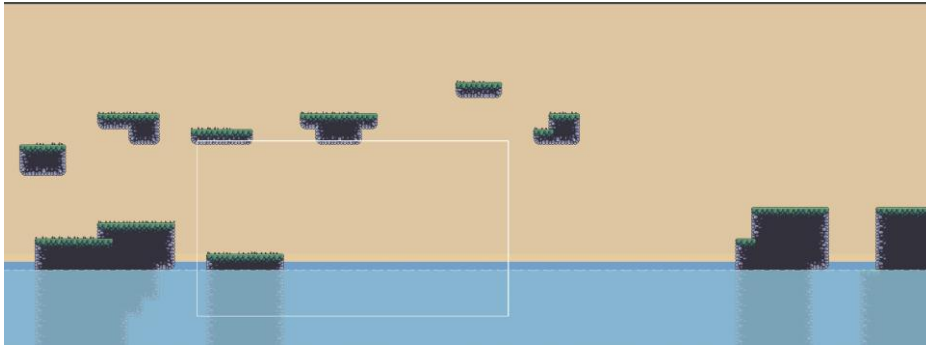
5. *Level 3:*



6. *Level 4:*



7. Level 5:



3.4.2. Player

Các hành vi của Player bao gồm:

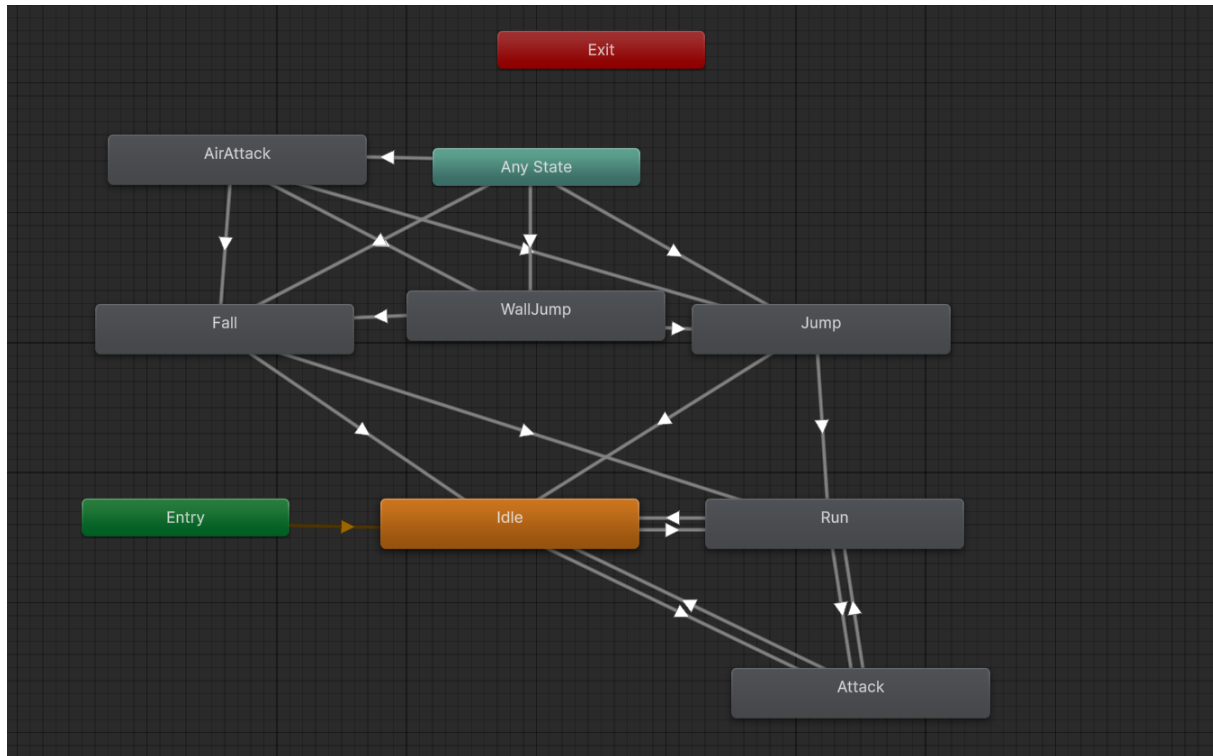
- **Di chuyển:** Player có thể di chuyển theo các hướng khác nhau thông qua các phím mũi tên hoặc tổ hợp phím **A, S, D**.
- **Nhảy:** Player thực hiện nhảy bằng cách nhấn phím **Space** khi ở trên mặt đất
- **Trượt tường:** Khi chạm tường mà không chạm đất, Player sẽ ở trạng thái trượt tường, trong đó vận tốc rơi bị hạn chế nhờ ma sát cao hơn so với rơi tự do.
- **Tấn công:** Nhấn phím **J** để Player tấn công, có thể thay đổi hướng di chuyển của kẻ thù và các viên đạn ngọc trai.
- **Nhận sát thương:** Player sẽ bị giảm máu khi va chạm với các vật thể gây sát thương.
- **Rơi xuống từ platform:** Player thực hiện hành vi bằng cách nhấn phím **S**.

Animation của Player là tập hợp các hành động thể hiện các trạng thái khác nhau của Player hoặc các đối tượng trong trò chơi. Chúng giúp Player trở nên sống động và tạo cảm giác tương tác hấp dẫn hơn với người chơi.

Các trạng thái Animation của Player bao gồm:

- **Idle:** Trạng thái khi Player đứng yên, không thực hiện bất kỳ hành động di chuyển nào.
- **Run:** Trạng thái khi Player di chuyển sang trái hoặc phải.
- **Jump:** Trạng thái khi Player nhảy lên khỏi mặt đất, với tọa độ y tăng dần.

- **WallJump:** Trạng thái khi Player thực hiện trượt trên tường.
- **Fall:** Trạng thái khi Player đang ở trên không và tọa độ y giảm xuống.
- **Attack:** Trạng thái tấn công khi Player đứng trên mặt đất.
- **AirAttack:** Trạng thái tấn công khi Player đang ở trên không.



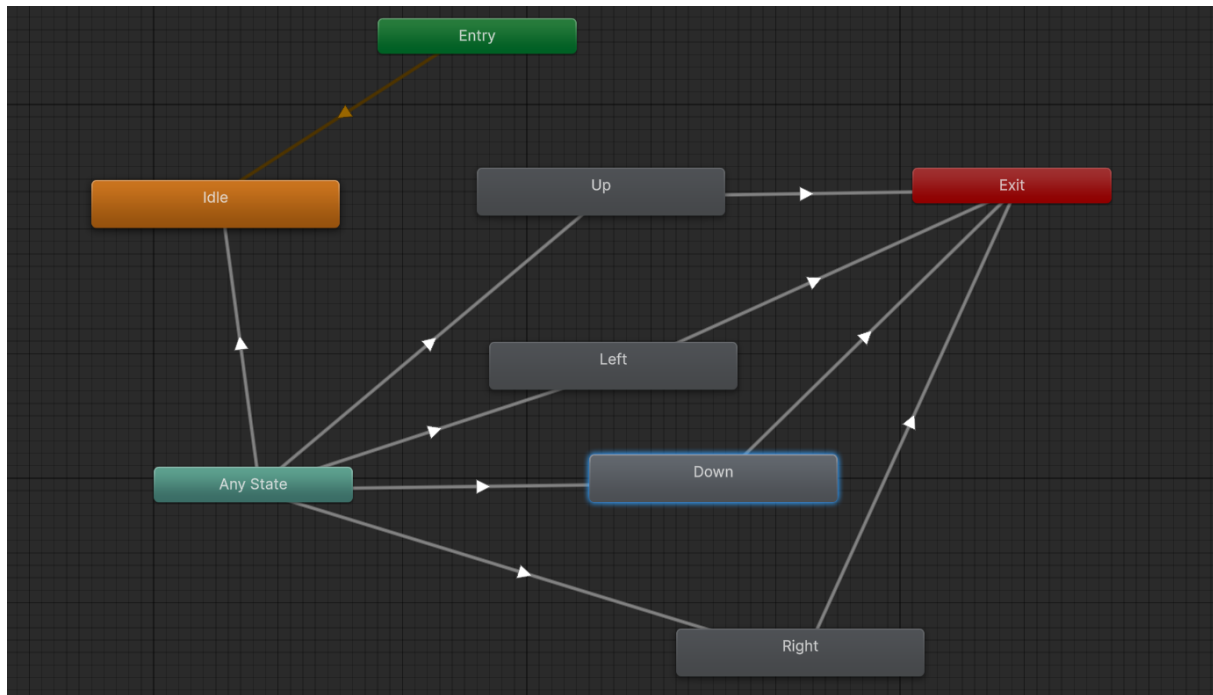
3.4.3. Icon Player

Trên màn hình Overworld, Player được đại diện bởi một icon di chuyển giữa các node, mỗi node tương ứng với một màn chơi. Khi hoàn thành màn chơi hiện tại, đường đi và màn chơi tiếp theo sẽ được mở khóa.

Người chơi có thể điều khiển icon Player bằng các phím mũi tên hoặc tổ hợp **W, A, S, D**, di chuyển theo hướng của node muốn đến. Để vào màn chơi tương ứng, người chơi đứng trên node đó và nhấn phím **Enter**.



Animation của icon:



3.4.4. Platform trong game

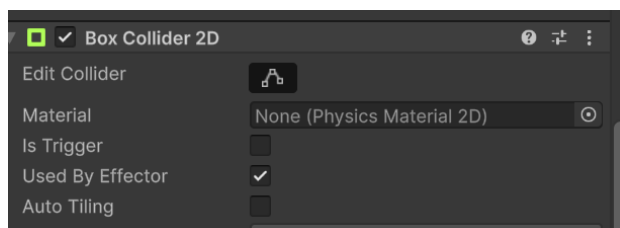
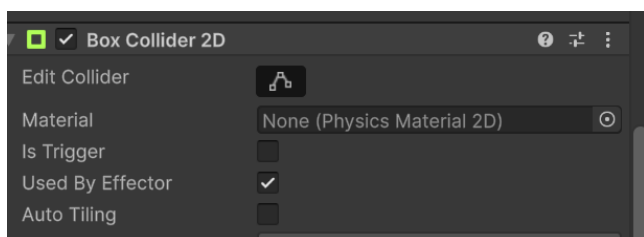
Platform trong trò chơi cho phép Player có thể nhảy lên từ dưới, đi qua platform, đồng thời cũng có thể rơi xuống khi thực hiện hành động rơi qua platform bằng cách nhấn phím S.

Trong **Unity**, cách hiệu quả để hiện thực hóa cơ chế này là sử dụng **Collider** kết hợp với **Platform Effector 2D**. Cách thực hiện như sau:

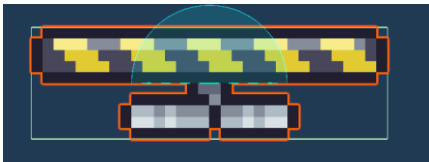
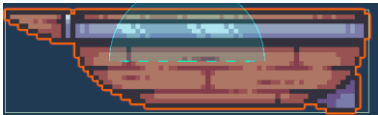
1. Thêm component **Collider** (thường là **Box Collider 2D**) vào platform.
2. Bật tùy chọn “**Used by Effector**” trong Collider để nó hoạt động cùng **Platform Effector 2D**.
3. Thêm component **Platform Effector 2D** vào platform.
4. Điều chỉnh các thông số trong **Platform Effector 2D** nếu cần:
 - **Use One Way**: Cho phép Player đi qua platform theo một chiều.
 - **Use One Way Grouping**: Giúp nhiều platform liên kế hoạt động như một thể thống nhất.

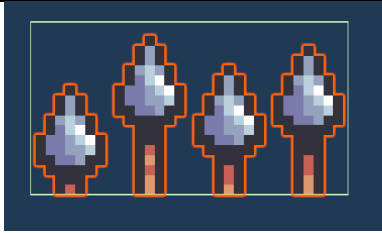
- **Use Side Friction / Use Side Bounce:** Điều chỉnh ma sát và độ nảy khi Player chạm vào cạnh của platform.

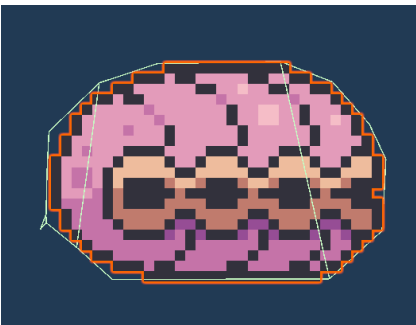
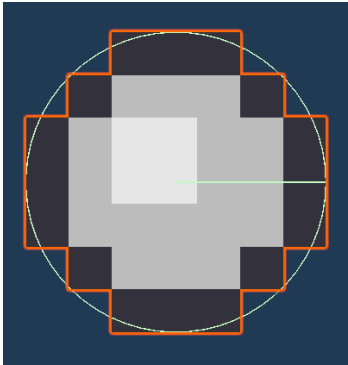
Cách kết hợp này đảm bảo Player có thể nhảy lên, đứng trên platform và rơi xuống một cách mượt mà, đồng thời giữ nguyên logic vật lý tự nhiên của trò chơi.

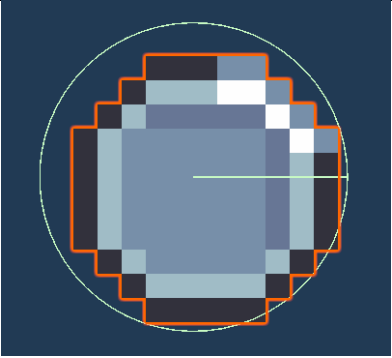
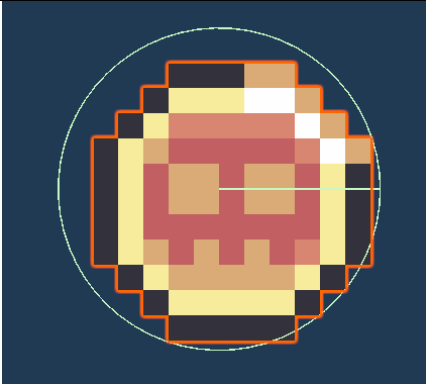
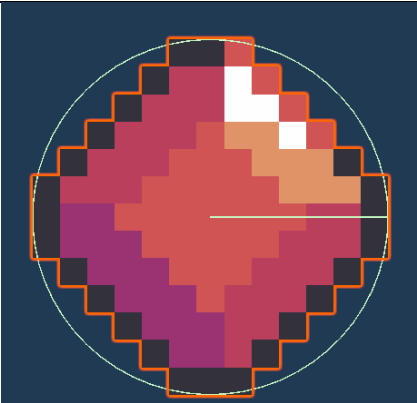
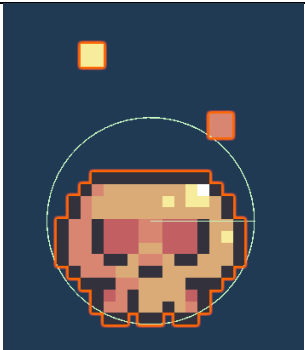


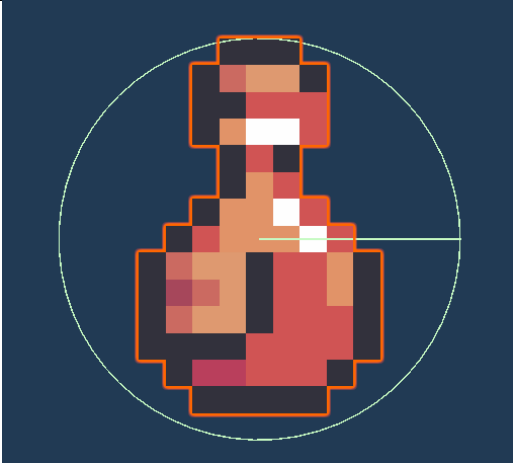
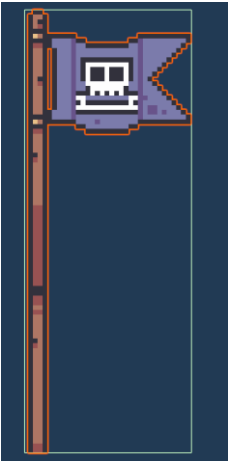
3.4.5. Một số object trong game

Tên Object	Hình ảnh	Đặc điểm
Helicopter		Là một Moving Platform , di chuyển theo phương ngang hoặc phương dọc trong một phạm vi xác định.
Boat		Là một Moving Platform , di chuyển trên mặt nước theo phương ngang trong một phạm vi nhất định; khi đổi hướng di chuyển, Boat cũng sẽ đổi hướng hiển thị.

Palms		Là 1 trong các Static Platform cố định trong trò chơi.
Floor Spike		Gậy sắt thương cho Player khi có va chạm.
Spike		Quay quanh một trục cố định theo các góc xác định. Khi Player chạm vào Spike Ball sẽ bị trừ máu.
Static Saw		Là một lưỡi cưa cố định, xoay tại chỗ ở một vị trí nhất định. Khi Player chạm vào sẽ bị trừ máu.
Saw		Di chuyển quanh một trục xác định trong một phạm vi cho trước. Khi Player tiếp xúc sẽ bị trừ máu.

Tooth		Là một Enemy, di chuyển liên tục trong một khu vực. Nếu Player va chạm sẽ bị trừ máu. Có thể dùng chiêu tấn công để thay đổi hướng di chuyển của Tooth.
Shell		Là một Enemy đứng yên. Khi phát hiện Player trong tầm ngắm, Shell sẽ bắn ra các viên đạn ngọc trai với tần suất 3 giây/lần.
Pearl		Được bắn ra bởi Shell, di chuyển theo vector ngang. Khi va chạm với tường hoặc Player sẽ biến mất và gây sát thương. Nếu không va chạm sau một khoảng thời gian, Pearl sẽ tự động biến mất. Có thể dùng chiêu tấn công để thay đổi hướng của Pearl.

Silver Coin		Item khi thu thập sẽ cộng thêm 1 Coin .
Gold Coin		Item khi thu thập sẽ cộng thêm 5 Coin .
Diamond		Item khi thu thập sẽ cộng thêm 20 Coin .
Skull		Item khi thu thập sẽ cộng thêm 50 Coin .

Potion		Item khi thu thập sẽ hồi phục 1 máu cho Player.
Flag		Là điểm kết thúc màn chơi . Khi Player chạm vào, sẽ được teleport về Overworld và mở khóa màn chơi tiếp theo.
Treasure		Vật phẩm đặc biệt xuất hiện ở màn cuối. Khi Player chạm vào, sẽ chiến thắng trò chơi và mở khóa kho báu hải tặc .

3.4.5. Hệ thống Âm thanh trong Game

a. Mục tiêu

Âm thanh được tích hợp để:

- Tăng trải nghiệm người chơi (thêm sự sinh động, kịch tính).
- Tạo phản hồi tức thì cho các hành động trong game (nhảy, bị tấn công, thu thập vật phẩm).

- Duy trì không khí cho từng màn chơi thông qua nhạc nền (Background Music).

b. Kiến trúc hệ thống âm thanh

Hệ thống âm thanh được xây dựng dựa trên mô hình **Event-based Audio System**, bao gồm:

1. EventManager

- Trung gian quản lý sự kiện.
- Các script gameplay không phát nhạc trực tiếp, mà chỉ gọi sự kiện (Trigger).

2. SoundType

- Liệt kê tất cả các loại âm thanh (nhạc nền, hiệu ứng, UI).

3. SoundEventData

- Lưu trữ dữ liệu cần thiết khi phát âm thanh (loại âm thanh, volume, loop, thời gian bắt đầu, v.v).

4. AudioManager

- Thành phần trung tâm xử lý âm thanh.
- Nhận sự kiện từ EventManager và phát nhạc/hiệu ứng phù hợp.

c. Danh sách âm thanh

Tên âm thanh	Loại	Mô tả
BGM_MainMenu	BGM	Nhạc nền khi ở màn hình chính
BGM_Level	BGM	Nhạc nền xuyên suốt màn chơi
SFX_Jump	SFX	Âm thanh khi player nhảy
SFX_Coin	SFX	Âm thanh khi nhặt coin
SFX_Damage	SFX	Âm thanh khi player bị trúng đòn

Tên âm thanh	Loại	Mô tả
SFX_ClickButton	UI	Âm thanh khi click vào nút
SFX_Pearl	SFX	Âm thanh khi Shell bắn Pearl
SFX_Victory	SFX	Âm thanh khi hoàn thành màn chơi

d. Code minh họa

- *EventManager.cs*

```
using System;
```

```
using System.Collections.Generic;
```

```
public static class EventManager
```

```
{
```

```
private static Dictionary<string, Action<object>> eventTable = new();
```

```
public static void Subscribe(string eventName, Action<object> listener)
```

```
{
```

```
if (!eventTable.ContainsKey(eventName))
```

```
eventTable[eventName] = listener;
```

```
else
```

```
eventTable[eventName] += listener;
```

```
}
```

```
public static void Unsubscribe(string eventName, Action<object> listener)
```

```

{
    if (eventTable.ContainsKey(eventName))
    {
        eventTable[eventName] -= listener;

        if (eventTable[eventName] == null)
            eventTable.Remove(eventName);
    }
}

```

```

public static void Trigger(string eventName, object param = null)
{
    if (eventTable.ContainsKey(eventName))
        eventTable[eventName]?.Invoke(param);
}
}

```

- ***SoundType.cs***

```

public enum SoundType
{
    BGM_MainMenu,

    BGM_Level,

    SFX_Jump,

    SFX_Coin,

```

```
SFX_Damage,  
  
SFX_ClickButton,  
  
SFX_Pearl,  
  
SFX_Victory,  
  
}
```

SoundEventData.cs

```
public class SoundEventData  
{  
    public SoundType Type;  
    public float Volume;  
    public bool Loop;  
    public float StartTime;  
  
    public SoundEventData(  
        SoundType type,  
        float volume = 1f,  
        bool loop = false,  
        float startTime = 0f)  
    {  
        Type = type;  
        Volume = volume;  
        Loop = loop;  
        StartTime = startTime;  
    }  
}
```

}

}

- **AudioManager.cs**

using UnityEngine;

public class AudioManager : MonoBehaviour

{

public static AudioManager Instance { get; private set; }

[SerializeField] private AudioSource musicSource;

[SerializeField] private AudioSource sfxSource;

[System.Serializable]

public class Sound

{

public SoundType type;

public AudioClip clip;

}

[SerializeField] private Sound[] sounds;

private void Awake()

{


```

        if (Instance == null) { Instance = this;
DontDestroyOnLoad(gameObject); }

        else { Destroy(gameObject); return; }

    }

private void OnEnable()

{

    EventManager.Subscribe("PlaySFX", OnPlaySFX);

    EventManager.Subscribe("PlayMusic", OnPlayMusic);

    EventManager.Subscribe("StopMusic", OnStopMusic);

}

private void OnDisable()

{

    EventManager.Unsubscribe("PlaySFX", OnPlaySFX);

    EventManager.Unsubscribe("PlayMusic", OnPlayMusic);

    EventManager.Unsubscribe("StopMusic", OnStopMusic);

}

private AudioClip GetClip(SoundType type)

{

    foreach (var s in sounds)

        if (s.type == type) return s.clip;

    return null;

```

```
}
```

```
private void OnPlayMusic(object data)
```

```
{
```

```
if (data is SoundEventData sound)
```

```
{
```

```
AudioClip clip = GetClip(sound.Type);
```

```
if (clip == null) return;
```

```
musicSource.clip = clip;
```

```
musicSource.volume = sound.Volume;
```

```
musicSource.loop = sound.Loop;
```

```
musicSource.time = sound.StartTime;
```

```
musicSource.Play();
```

```
}
```

```
}
```

```
private void OnStopMusic(object data)
```

```
{
```

```
musicSource.Stop();
```

```
}
```

```
private void OnPlaySFX(object data)
```

```
{
```

```

        if (data is SoundEventData sound)
        {
            AudioClip clip = GetClip(sound.Type);

            if (clip == null) return;

            sfxSource.PlayOneShot(clip, sound.Volume);
        }
    }
}

```

e. Ví dụ sử dụng trong gameplay

- **Phát nhạc nền khi vào màn chơi:**

```

EventManager.Trigger("PlayMusic", new
    SoundEventData(SoundType.BGM_Level, 0.5f, true));

```

- **Phát âm thanh khi nhảy:**

```

EventManager.Trigger("PlaySFX", new SoundEventData(SoundType.SFX_Jump, 1f));

```

- **Phát âm thanh khi nhặt coin:**

```

EventManager.Trigger("PlaySFX", new SoundEventData(SoundType.SFX_Coin, 1f));

```

- **Phát âm thanh khi hoàn thành màn chơi:**

```

EventManager.Trigger("PlaySFX", new SoundEventData(SoundType.SFX_Victory,
    1f));

```

f. Ưu điểm của hệ thống

- **Tách biệt** gameplay và âm thanh → dễ bảo trì, dễ mở rộng.
- **Quản lý tập trung:** điều chỉnh được nhạc nền, hiệu ứng theo nhóm.

- **Linh hoạt:** chỉ cần gọi event, không cần quan tâm chi tiết AudioSource.
- **Tăng trải nghiệm:** âm thanh rõ ràng, đồng bộ với hành động trong game.

3.4.6. Quản lý Scene bằng Enum và Extension

Trong Unity, việc load scene thông thường đòi hỏi phải truyền trực tiếp tên scene dưới dạng chuỗi (string). Cách làm này có nhược điểm là dễ gây lỗi khi gõ sai tên, hoặc khi đổi tên scene trong Build Settings.

Để khắc phục, hệ thống sử dụng một **enum GameScene** liệt kê tất cả các scene trong game. Mỗi giá trị trong enum sẽ ánh xạ đến đúng tên scene thực tế thông qua một **phương thức mở rộng (extension method) GetSceneName()**.

Ví dụ:

```
public enum GameScene
```

```
{
```

```
    Overworld,
```

```
    Level0,
```

```
    Level1,
```

```
    Level2,
```

```
    Level3,
```

```
    Level4,
```

```
    Level5,
```

```
}
```

```
public static class GameSceneExtensions
```

```
{
```

```
    public static string GetSceneName(this GameScene scene) =>
```

```
        scene switch
```

```

{
    GameScene.Overworld => "Overworld",

    GameScene.Level0    => "OmniScene",

    GameScene.Level1    => "1",

    GameScene.Level2    => "2",

    GameScene.Level3    => "3",

    GameScene.Level4    => "4",

    GameScene.Level5    => "5",

    _ => throw new ArgumentOutOfRangeException(nameof(scene), scene, null)

};
}

```

Khi cần load scene, thay vì viết trực tiếp tên chuỗi, ta chỉ cần gọi:

```
StartCoroutine(LoadSceneDelayed(currentNode.scene.GetSceneName(), 0.2f));
```

Trong đó:

- `currentNode.scene` là một giá trị của enum `GameScene`.
- `GetSceneName()` sẽ trả về tên scene chính xác theo Build Settings.
- `LoadSceneDelayed()` là coroutine chịu trách nhiệm load scene có độ trễ (ví dụ 0.2s).

Ưu điểm

- **Hạn chế lỗi:** không còn phải nhớ hoặc gõ lại tên scene dạng chuỗi.
- **Dễ bảo trì:** nếu đổi tên scene, chỉ cần sửa trong `GameSceneExtensions`.
- **Code rõ ràng:** khi đọc code, việc sử dụng `GameScene.Level1` sẽ trực quan hơn nhiều so với "1".

3.4.7. Camera

Trong quá trình di chuyển, nhân vật cần luôn nằm trong khung hình để người chơi quan sát dễ dàng. Để hiện thực điều này, hệ thống sử dụng **Cinemachine**, một công cụ mạnh mẽ hỗ trợ quản lý camera trong Unity.

- **Cinemachine Virtual Camera:** được thêm vào scene và thiết lập **Tracking Target** là player. Nhờ đó, camera sẽ tự động theo dõi và dịch chuyển mượt mà theo vị trí nhân vật. Ngoài ra, thông số **Lens** cũng được điều chỉnh để có tầm nhìn bao quát phù hợp với từng màn chơi.
- **Cinemachine Confiner 2D:** để tránh tình trạng camera trượt ra khỏi bản đồ khi nhân vật đi sát biên, thành phần Confiner 2D được sử dụng để xác định ranh giới chuyển động cho camera. Điều này đảm bảo khung hình luôn nằm trong vùng gameplay và không để lộ các khoảng trống ngoài bản đồ.

Với cách thiết lập trên, camera không chỉ luôn bám sát chuyển động của người chơi mà còn tạo cảm giác liền mạch, mang lại trải nghiệm trực quan và chuyên nghiệp hơn.

3.4.8. UI

Mô tả nhiệm vụ:

Giao diện người dùng (UI – User Interface) trong game được thiết kế nhằm hỗ trợ người chơi theo dõi thông tin cần thiết trong suốt quá trình trải nghiệm, đồng thời cung cấp các màn hình điều hướng như menu chính, kết thúc màn chơi và bản đồ chọn màn (overworld). UI được xây dựng đơn giản, trực quan và đồng bộ với phong cách hình ảnh của trò chơi.

Cách thực hiện:

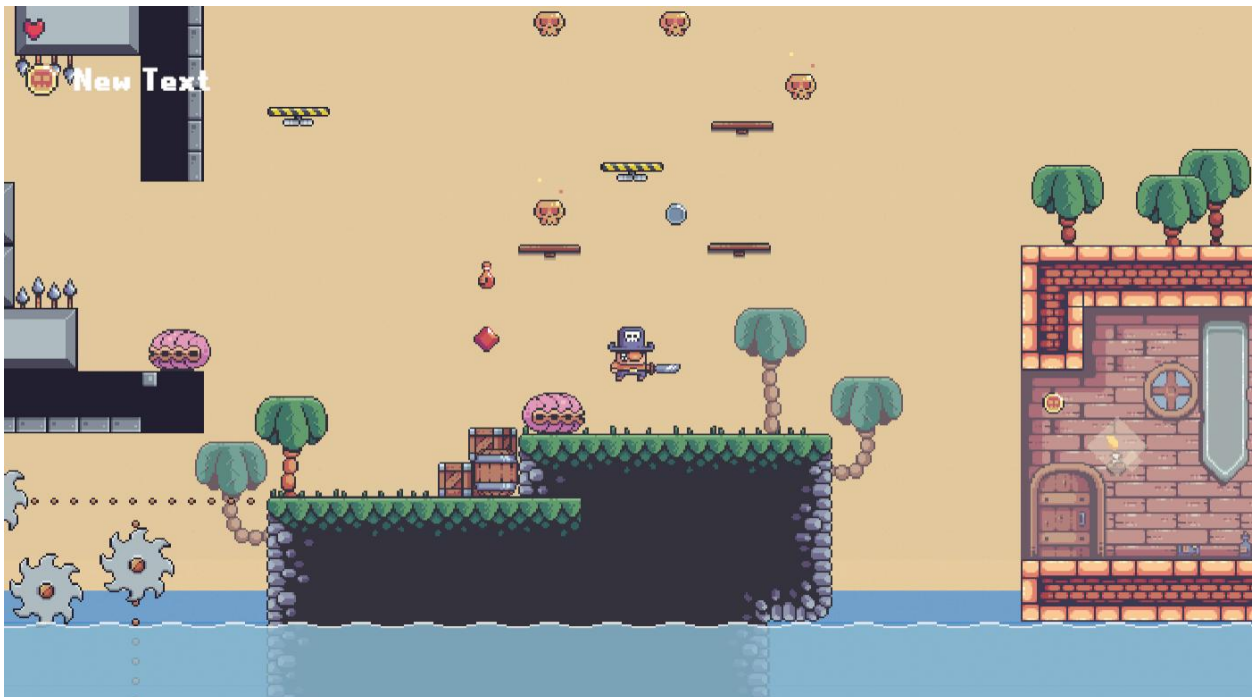
1. Menu chính (Main Menu):

- Là màn hình khởi đầu của trò chơi.
- Bao gồm các nút chức năng cơ bản:
 - Play: bắt đầu trò chơi vào màn hình Overworld
 - Exit: thoát khỏi trò chơi.
- Menu được thiết kế tối giản để người chơi dễ dàng tiếp cận.



2. UI trong màn chơi (In-Game UI):

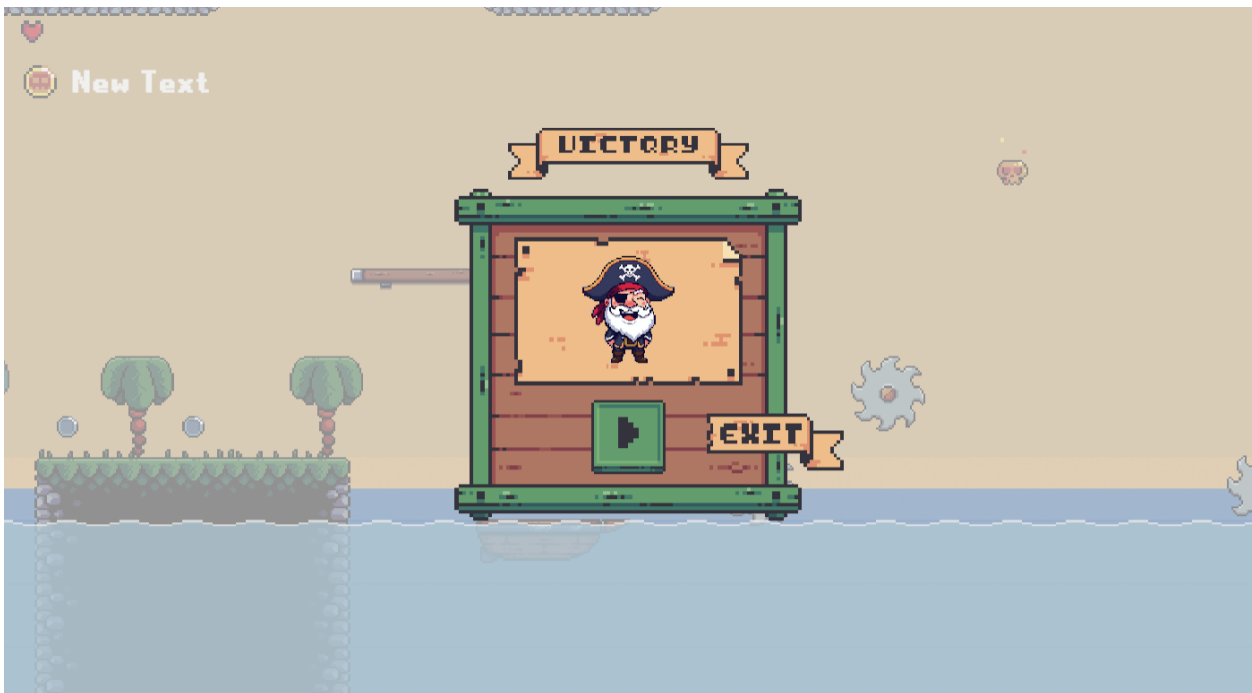
- Điểm số (Score): hiển thị số Coin mà người chơi thu thập được.
- Mạng sống/Health: thể hiện số lần nhân vật có thể chịu sát thương.
- Các thành phần UI được bố trí gọn gàng ở góc màn hình.



3. UI thắng/thua cuộc (End Screen):

- Khi thắng màn chơi, UI hiển thị thông báo chúc mừng, điểm số đạt được và nút Next để tiếp tục sang màn mới, hoặc nút Exit thoát trò chơi.

- Khi thua màn chơi, UI hiển thị thông báo Game Over cùng các lựa chọn Replay (chơi lại) hoặc Exit (thoát trò chơi).



4. UI overworld hiển thị level:

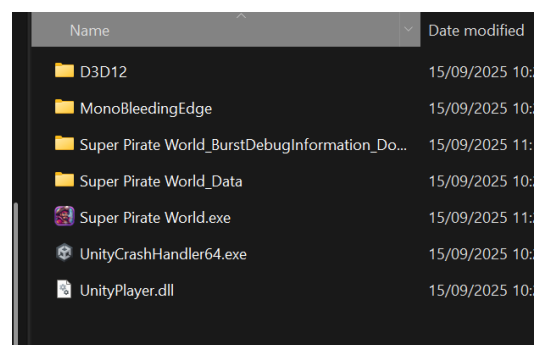
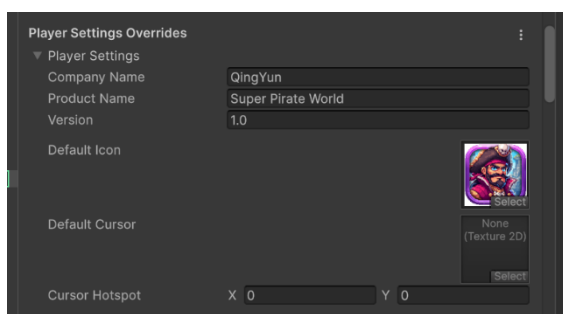
- Overworld đóng vai trò như bản đồ tổng quan, nơi người chơi chọn màn chơi.
- UI hiển thị Level hiện tại của Player



3.4.9. Build game

Sau khi hoàn thiện dự án, Unity cung cấp chức năng **Build** để xuất trò chơi thành ứng dụng có thể chạy trên nhiều nền tảng. Quy trình build trên Windows được thực hiện như sau:

1. Vào menu **File** → **Build Settings**.
2. Trong danh sách nền tảng, chọn **Windows**.
3. Cấu hình các tùy chọn build như:
 - **Tên ứng dụng (Product Name)**
 - **Biểu tượng ứng dụng (Icon)**
 - **Phiên bản ứng dụng (Version)**
4. Nhấn **Build** và chọn thư mục lưu trữ. Unity sẽ tạo ra các tệp cần thiết để chạy trò chơi trên nền tảng đã chọn.



IV. NHẬN XÉT – ĐÁNH GIÁ – KIẾN NGHỊ

4.1. Những điều học được sau thực tập

Qua quá trình thực tập, em đã tiếp thu được nhiều kiến thức và kỹ năng thực tế bên cạnh những kiến thức đã học ở trường. Cụ thể:

- Hiểu rõ hơn về **quy trình phát triển phần mềm/game** từ khâu phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến build sản phẩm.
- Nắm bắt và sử dụng thành thạo **Unity Engine**, đặc biệt là các module quan trọng như **UI, âm thanh, camera, quản lý scene, animation** và **xử lý sự kiện**.
- Rèn luyện kỹ năng **làm việc nhóm, giao tiếp và quản lý thời gian** trong môi trường thực tế.
- Tăng khả năng **tự học và giải quyết vấn đề** khi gặp lỗi phát sinh trong quá trình code và build dự án.

4.2. Điểm mạnh và điểm yếu bản thân

- **Điểm mạnh:**
 - Có tinh thần ham học hỏi, chịu khó tìm tòi tài liệu để giải quyết vấn đề.
 - Có khả năng lập trình ở mức cơ bản đến trung bình khá, biết vận dụng kiến thức để xây dựng các chức năng cần thiết.
 - Thái độ làm việc nghiêm túc, trách nhiệm trong quá trình thực hiện nhiệm vụ được giao.
- **Điểm yếu:**
 - Kinh nghiệm thực tế còn hạn chế, đôi khi mất nhiều thời gian để xử lý lỗi hoặc tối ưu code.
 - Khả năng quản lý tiến độ công việc chưa thật sự tốt, đôi khi bị trễ so với kế hoạch ban đầu.
 - Kỹ năng giao tiếp và làm việc nhóm còn cần cải thiện để phối hợp hiệu quả hơn.

4.3. Góp ý cải tiến cho công ty

Trong thời gian thực tập, em nhận thấy công ty đã tạo điều kiện rất tốt để sinh viên có môi trường học hỏi. Tuy nhiên, em xin phép đưa ra một số góp ý nhỏ:

- Có thể tổ chức thêm các **buổi hướng dẫn chuyên sâu** về công nghệ cụ thể (Unity, C#, thiết kế game, tối ưu hiệu năng).
- Tăng cường **trao đổi, phản hồi định kỳ** để thực tập sinh dễ dàng nắm bắt mức độ hoàn thành công việc.

4.4. Hướng phát triển đề tài/sản phẩm thực tập sau này

Trong tương lai, em mong muốn phát triển đề tài thực tập theo các hướng sau:

- Hoàn thiện thêm các **màn chơi mới** với độ khó tăng dần nhằm tạo sự hấp dẫn cho người chơi.
- Nâng cấp **UI/UX** để trải nghiệm người dùng trở nên trực quan và sinh động hơn.
- Tích hợp **hệ thống lưu tiến trình** (save/load game) để người chơi có thể tiếp tục ở vị trí đã dừng.
- Mở rộng sản phẩm sang **nhiều nền tảng khác nhau** (Android, iOS) để tăng tính ứng dụng.

V. TÀI LIỆU THAM KHẢO

[1] Unity Technologies, *Unity Documentation*, <https://docs.unity.com>.

[2] Microsoft, *C# Documentation*, <https://learn.microsoft.com/en-us/dotnet/csharp/>.

[3] Brackeys, *Unity 2D Game Development Tutorials*,
<https://www.youtube.com/user/Brackeys>.

[4] W3Schools, *C# Tutorial*, <https://www.w3schools.com/cs/>.

[5] Studocu, *Báo cáo thực tập ngành CNTT – Trường Đại học Công nghệ và Quản lý Hữu nghị*, <https://www.studocu.vn/vn/document/truong-dai-hoc-cong-nghe-va-quan-ly-huu-nghi/phuong-phap-nghien-cuu/bao-cau-thuc-tap-nganh-cntt/52330703>.

[6] Clear Code, *Creating an amazing 2D platformer in Python [SNES inspired]*,
<https://youtu.be/WViyCAa6yLI?si=vSamm0Gi3S24kVSC>.